

PROJECT VAULT

Product Requirements Document

Version 1.0 · For Claude Code

A context-aware AI workspace for managing projects, files, chat history, and AI-assisted workflows.

Table of Contents

1. Overview & Vision
2. Tech Stack & Architecture
3. Module 1 — Settings & Appearance
4. Module 2 — Chat Interface & Voice
5. Module 3 — Project Context Engine
6. Module 4 — Search
7. Module 5 — Email Integration
8. Module 6 — Export
9. Module 7 — PDF Engine
10. Module 8 — Image AI (Gemini Nano)
11. Data Models
12. Local Development, GitHub & Railway Deployment
13. File & Folder Structure
14. Environment Variables
15. Acceptance Criteria

1. Overview & Vision

Project Vault is a single-page React application that acts as a personal AI workspace. Users manage long-running projects, attach files, define rich context fields, and chat with an AI assistant that is aware of that context. Every session persists to a backend so nothing is lost.

Core Principles

- Context-first — Claude always knows which project is active and uses its context fields automatically.
 - Voice-native — all responses can be read aloud; all prompts can be spoken.
 - Export everything — chats, projects, and files must be portable (PDF, JSON, email).
 - Visual flexibility — users control fonts, colours, and icon packs from a Settings page.
 - AI-augmented documents — PDFs can be uploaded, summarised, and extracted; images can be generated and interpreted via Gemini Nano.
-

2. Tech Stack & Architecture

Frontend

- React 18 + Vite
- Tailwind CSS — utility classes only, no custom build step needed
- DM Sans (UI font) + DM Mono (code/mono font) from Google Fonts
- lucide-react for icons (swappable — see Module 1)
- React Router v6 for page routing
- Zustand for global state (projects, settings, chat history)

Backend

- Node.js + Express REST API
- SQLite via better-sqlite3 for persistence (file: ./data/vault.db)
- Multer for file upload handling
- Nodemailer for email dispatch
- pdf-lib + pdfjs-dist for PDF creation and extraction

AI Services

- Anthropic Claude API — chat completions (claude-sonnet-4-20250514)
- Gemini Nano (window.ai / Chrome Built-in AI) — image generation & interpretation
- Web Speech API — speech-to-text (microphone input) and text-to-speech (response playback)

Environment Variables (.env)

```
ANTHROPIC_API_KEY=sk-ant-...
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=you@gmail.com
SMTP_PASS=app-password
PORT=3001
```

3. Module 1 — Settings & Appearance

Route

/settings

Sub-sections

3.1 Font Selection

Present a dropdown or visual picker for UI font. Options:

- DM Sans (default — matches Claude workspace)
- Inter
- Geist
- Lora (serif option)
- JetBrains Mono (developer option)

Fonts are loaded dynamically from Google Fonts on selection. Store choice in localStorage under vault_settings.font.

3.2 Colour Theme

Provide at minimum the following named themes:

Theme Name	Background	Accent
Warm Sand (default)	#F5F5F0	#CC785C
Dark Slate	#0F1117	#7C8CF8
Forest	#F0F4F0	#3D7A5E
Midnight Blue	#0D1B2A	#5DADE2
Paper White	#FFFFFF	#333333

Inject selected theme colours as CSS custom properties on :root so all components respond instantly. Store in localStorage under vault_settings.theme.

3.3 Icon Pack

Default: lucide-react. Allow switching between:

- Lucide (default)
- Heroicons
- Phosphor Icons

Implement an IconProvider context that maps semantic icon names (e.g. icon('folder'), icon('mic')) to the selected library. All components consume IconProvider — never import icon libraries directly.

3.4 Persistence

All settings are saved to localStorage immediately on change. On app load, read settings and apply before first render to avoid flash of unstyled content.

4. Module 2 — Chat Interface & Voice

Route

```
| /chat or /projects/:id/chat
```

Chat UI Requirements

- Full-height message thread with user and assistant bubbles
- Markdown rendering in assistant messages (use react-markdown + remark-gfm)
- Code blocks with syntax highlighting (use react-syntax-highlighter)
- Streaming responses — use the Anthropic streaming API and display tokens as they arrive
- Message timestamps shown on hover
- Copy-to-clipboard button on each assistant message

4.1 Project Context Injection

When a project is active (selected from sidebar), prepend a system prompt to every Claude API call:

```
You are helping with the project: {project.name}.  
Goal: {project.goal}  
Problem: {project.problem}  
Audience: {project.audience}  
Tech Stack: {project.techStack}  
Constraints: {project.constraints}  
Definition of Done: {project.successCriteria}  
Tone: {project.tone}  
Additional context: {project.notes}  
Files attached: {project.files.map(f => f.name).join(', ')}
```

This system prompt is built in a `useSystemPrompt(projectId)` hook and passed as the `system` field of each API request.

4.2 Voice — Microphone Input (STT)

- Show a mic button (icon: Mic) in the chat input bar
- On press: start Web Speech API `SpeechRecognition` session
- Continuous listening with interim results shown in the input field
- On release or silence: commit transcript to the input field
- Show animated waveform while listening
- Graceful fallback message if browser does not support Web Speech API

4.3 Voice — Read Aloud (TTS)

- Show a speaker button (icon: Volume2) on every assistant message
- On click: use `window.speechSynthesis.speak()` with the message text
- Auto-select a natural voice: prefer en-GB or en-AU female voices
- Show pause/stop controls while speaking
- Allow user to select preferred voice in Settings > Voice
- Rate and pitch sliders in Settings (default: rate 1.0, pitch 1.0)

5. Module 3 — Project Context Engine

Route

```
/projects (list)
/projects/:id (detail / editor)
```

5.1 Project Data Fields

Field	Type	Purpose for Claude
name	string	Project identifier
goal	string	Objective / desired outcome
problem	string	Pain point being solved
audience	string	Who will use or read this
techStack	string	Technologies in play
constraints	string	Limits, deadlines, must-avoid
successCriteria	string	Definition of done
tone	string	Preferred response style

notes	string	Freeform Claude context
files[]	FileRecord[]	Attached file metadata
createdAt	ISO string	Audit trail
updatedAt	ISO string	Cache invalidation

5.2 Reference Material — "Ask about a project"

In the chat input bar, add a @ mention dropdown. Typing @ opens a list of all projects. Selecting one injects a reference block into the system prompt:

```
--- REFERENCE PROJECT: {project.name} ---  
{all context fields}  
--- END REFERENCE ---
```

Multiple projects can be referenced in a single conversation. The hook `useReferencedProjects()` tracks which projects are active references for the current chat session.

5.3 File Storage

- Files are uploaded via POST `/api/projects/:id/files` (multipart/form-data)
- Stored on disk under `./uploads/{projectId}/`
- File metadata (name, size, mimetype, path, uploadedAt) stored in SQLite
- Files can be downloaded via GET `/api/files/:fileId`
- Files are included by name in the Claude system prompt so Claude is aware of them

6. Module 4 — Search

Global Search (across all projects)

- Keyboard shortcut: Cmd+K / Ctrl+K opens a modal search palette
- Search across: project names, all context field text, file names, chat message content
- Results grouped by type: Projects, Files, Messages
- Click a result to navigate to that project or jump to that message in chat
- Implemented with SQLite FTS5 virtual table for full-text search

In-Project Search

- Search icon in the project detail header
- Searches only within the active project: context fields, file names, chat messages
- Highlights matching text inline

Backend

```
GET /api/search?q={query}&scope=global
GET /api/search?q={query}&scope=project&projectId={id}
```

Use SQLite FTS5. Index columns: project name, all text context fields, chat message content, file names.

7. Module 5 — Email Integration

"Email this thread" feature

- Button in chat header: icon Mail, label "Email thread"
- Opens a modal with: To (text input), Subject (pre-filled with project name + date), Body preview (rendered HTML of the conversation)
- Format: alternating user/assistant rows with timestamps, project context summary at the top
- Send via POST `/api/email/thread`

"Email project summary" feature

- Button on project detail page: icon SendHorizontal, label "Email summary"
- Compiles all context fields into a formatted HTML email
- Attaches file list as a text appendix

Backend — POST `/api/email/thread`

```
Body: { to, subject, messages: Message[], projectId? }
Build HTML template with inline CSS (email-safe)
Send via Nodemailer using SMTP env vars
Return { success: true } or { error: string }
```

8. Module 6 — Export

Export Chat as PDF

- Button: icon Download, label "Export chat"
- Calls POST `/api/export/chat` with { messages, projectName }
- Server generates PDF using pdf-lib: header with project name, timestamped messages, assistant messages in a lightly shaded box
- Returns binary PDF, browser triggers download

Export Chat as JSON

- Button: icon FileJson, label "Export JSON"

- Client-side: `JSON.stringify({ project, messages }, null, 2)`
- Triggers blob download

Export Project as JSON

- Button on project page: "Export project"
- Calls `GET /api/projects/:id/export`
- Returns full project object including file metadata (not file binaries)

Export All Projects

- Settings page: "Export all projects" button
 - Calls `GET /api/projects/export-all`
 - Returns JSON array of all projects
-

9. Module 7 — PDF Engine

9.1 Upload & Read PDF

- Users can upload PDF files to a project via drag-and-drop or file picker
- On upload, call `POST /api/pdf/extract` with the file
- Server uses `pdfjs-dist` to extract text from all pages
- Extracted text stored in SQLite alongside the file record
- Extracted text shown in a collapsible panel on the project file card

9.2 AI-Powered PDF Summary

- Button on each PDF file card: "Summarise with Claude"
- Sends extracted text to Claude API with prompt: Summarise this document concisely. Extract key facts, decisions, and action items.
- Summary stored against the file record and shown below the file card

9.3 Chat with a PDF

- Button: "Chat about this file"
- Opens chat session with extracted PDF text injected into system prompt
- User can ask questions, the assistant answers using document content

9.4 Generate PDF

- From Export module: create PDF from chat or project data (see Module 6)
- Also: a standalone "Create PDF report" button on project page

- Generates a formatted report with: cover page (project name, date), context fields, file list, optional latest chat transcript
 - Built with pdf-lib on the server, downloaded by the client
-

10. Module 8 — Image AI (Gemini Nano)

Availability Check

On app load, check if `window.ai` and `window.ai.createImageSession` are available (Chrome 127+ with Gemini Nano enabled). If not, hide image AI features and show a subtle info banner: "Image AI requires Chrome with Gemini Nano enabled."

10.1 Generate Image

- Button in chat input bar: icon `ImagePlus`, label "Generate image"
- Opens a small prompt input modal
- On submit: call `window.ai.createImageSession()` then `session.prompt(userPrompt)`
- Display the returned image blob inline in the chat message
- Image saved as a file attached to the active project

10.2 Interpret / Describe Image

- When a user uploads an image (jpg, png, webp, gif) to a project or drags one into chat:
- Offer button: "Describe with Gemini"
- Call `window.ai.createMultimodalSession()` and pass the image
- Display the description as an assistant message in the chat thread
- Store description in SQLite alongside the file record

10.3 Image in PDF / Export

- When generating a project PDF report, optionally include AI-generated images or image descriptions as captions
-

11. Data Models (SQLite)

projects

```
id TEXT PRIMARY KEY
name TEXT
goal TEXT, problem TEXT, audience TEXT, techStack TEXT
constraints TEXT, successCriteria TEXT, tone TEXT, notes TEXT
```

```
createdAt TEXT, updatedAt TEXT
```

files

```
id TEXT PRIMARY KEY
projectId TEXT REFERENCES projects(id)
name TEXT, size INTEGER, mimetype TEXT, path TEXT
extractedText TEXT    -- PDF text content
aiDescription TEXT    -- Gemini image description
aiSummary TEXT        -- Claude PDF summary
uploadedAt TEXT
```

messages

```
id TEXT PRIMARY KEY
sessionId TEXT        -- groups messages in one chat session
projectId TEXT REFERENCES projects(id)
role TEXT             -- 'user' | 'assistant' | 'system'
content TEXT
createdAt TEXT
```

FTS5 virtual table

```
CREATE VIRTUAL TABLE search_index USING fts5(
  rowid, type, projectId, title, body
);
```

Populate on every insert/update to projects and messages. type is 'project', 'message', or 'file'.

12. Local Development, GitHub & Railway Deployment

This application must run locally during development, be version-controlled on GitHub, and deploy to Railway with a single git push. The same codebase serves all three environments — only environment variables change between them.

12.1 Running Locally

Prerequisites

- Node.js 20+ (check: `node -v`)
- npm 9+ (check: `npm -v`)
- Git
- Chrome 127+ for Gemini Nano image features (optional)

First-time setup

```
git clone https://github.com/YOUR_USERNAME/project-vault.git
cd project-vault
cp .env.example .env      # fill in ANTHROPIC_API_KEY and SMTP values
npm install                # installs root, client, and server dependencies
npm run dev                # starts both servers concurrently
```

npm run dev launches two processes via the concurrently package:

- Vite dev server → http://localhost:5173 (React client with HMR)
- Express server → http://localhost:3001 (REST API + file handling)

Add this to client/vite.config.js so the frontend proxies all /api/* calls to Express — no hard-coded URLs needed anywhere in the client:

```
server: {
  proxy: { '/api': 'http://localhost:3001' }
}
```

Local data persistence

- SQLite database auto-created at ./data/vault.db on first run — no setup required
- Uploaded files stored in ./uploads/{projectId}/
- Both ./data/ and ./uploads/ are gitignored and never committed
- All CREATE TABLE statements must use IF NOT EXISTS — a fresh clone must work immediately without any manual database setup

12.2 GitHub Repository Setup

.gitignore — must include these entries

```
.env
node_modules/
data/          # SQLite database — never commit
uploads/       # User-uploaded files — never commit
dist/          # Vite production build output
.DS_Store
```

.env.example — commit this file

Commit a .env.example with every required key listed but no real values. Anyone cloning the repo sees exactly what to fill in:

```
ANTHROPIC_API_KEY=your_key_here
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_USER=your_email@gmail.com
SMTP_PASS=your_app_password
```

```
PORT=3001
UPLOAD_DIR=./uploads
DB_PATH=./data/vault.db
NODE_ENV=development
```

package.json scripts (root level)

Script	Command	Used by
dev	concurrently 'npm run dev:server' 'npm run dev:client'	Local development
dev:server	nodemon server/index.js	Local dev — server only
dev:client	vite client	Local dev — client only
build	vite build client --outDir ../dist	Railway build step
start	node server/index.js	Railway start command

Recommended branch strategy

- main — production-ready; every push auto-deploys to Railway
- dev — active development; merge into main when a feature is stable
- feature/* — individual features; PR into dev, never directly into main

12.3 Deploying to Railway

Single-service architecture

Deploy as one Railway service. In production, Express serves the Vite-built client as static files AND handles all /api/* routes on the same port. One service, one domain, minimal cost.

Express static file serving — required for production

Add this block to server/index.js. It must come after all API route registrations:

```
if (process.env.NODE_ENV === 'production') {
  const distPath = path.join(__dirname, '../dist');
  app.use(express.static(distPath));
  // Catch-all: send index.html for any non-API route (React Router)
  app.get('*', (req, res) => {
    res.sendFile(path.join(distPath, 'index.html'));
  });
}
```

railway.toml — commit to repo root

```
[build]
builder = "nixpacks"
buildCommand = "npm install && npm run build"

[deploy]
startCommand = "npm start"
healthcheckPath = "/api/health"
restartPolicyType = "on_failure"
```

Health check endpoint — Railway requires this

```
// server/routes/health.js
router.get('/health', (req, res) => res.json({ ok: true, ts: Date.now() }));
```

Persistent storage on Railway — volumes

Railway's default filesystem is ephemeral and resets on redeploy. Attach a Railway Volume for both uploads and the SQLite database:

- In Railway dashboard: service → Storage → Add Volume
- Mount path: /app/data (for SQLite) and /app/uploads (for files)
- Set env vars in Railway: DB_PATH=/app/data/vault.db and UPLOAD_DIR=/app/uploads
- No code changes needed — server/index.js already reads these from process.env

Setting environment variables on Railway

- Railway dashboard → your project → service → Variables tab
- Add every key from .env.example with real production values
- Set NODE_ENV=production — this triggers Express static file serving
- Railway injects variables at runtime; .env file is not used in production

Deployment workflow — after initial setup

```
git add .
git commit -m 'feat: describe your change'
git push origin main
```

Railway detects the push, runs npm install && npm run build, then npm start. The new version is live within ~60 seconds. No manual steps required.

Custom domain (optional)

- Railway auto-assigns a free *.up.railway.app subdomain on first deploy
- Custom domain: Railway dashboard → Settings → Domains → Add Domain
- Point a CNAME record at the Railway-provided target URL
- Railway handles HTTPS/TLS automatically

13. File & Folder Structure

```
project-vault/
├── client/                                # React + Vite frontend
│   ├── src/
│   │   ├── components/                  # Shared UI components
│   │   ├── pages/                      # Route-level components
│   │   │   ├── ProjectList.jsx
│   │   │   ├── ProjectDetail.jsx
│   │   │   ├── ChatPage.jsx
│   │   │   └── SettingsPage.jsx
│   │   ├── hooks/                      # Custom React hooks
│   │   │   ├── useSystemPrompt.js
│   │   │   ├── useVoice.js
│   │   │   ├── useSearch.js
│   │   │   └── useGeminiNano.js
│   │   ├── store/                      # Zustand stores
│   │   ├── providers/                  # IconProvider, ThemeProvider
│   │   └── utils/                      # Formatters, exporters
│   └── vite.config.js                  # Proxy /api → localhost:3001 in dev
├── server/                             # Node + Express backend
│   ├── routes/
│   │   ├── health.js                  # GET /api/health - Railway healthcheck
│   │   ├── projects.js
│   │   ├── files.js
│   │   ├── chat.js
│   │   ├── email.js
│   │   ├── export.js
│   │   ├── pdf.js
│   │   └── search.js
│   ├── db.js                          # SQLite setup & IF NOT EXISTS migrations
│   └── index.js                       # Express app + static serving in prod
├── uploads/                           # gitignored - Railway Volume in prod
├── data/                              # gitignored - Railway Volume in prod
├── dist/                              # gitignored - Vite build output
├── .env                               # gitignored - never commit
├── .env.example                       # committed - template for all env vars
├── .gitignore
├── railway.toml                       # committed - Railway build & deploy config
└── package.json                       # scripts: dev, build, start
```

14. Environment Variables

Variable	Required	Notes
----------	----------	-------

ANTHROPIC_API_KEY	Yes	Anthropic Console — claude-sonnet-4-20250514
SMTP_HOST	Yes (email)	e.g. smtp.gmail.com
SMTP_PORT	Yes (email)	587 for TLS
SMTP_USER	Yes (email)	Sending email address
SMTP_PASS	Yes (email)	App password, not account password
NODE_ENV	Yes in prod	Set to 'production' in Railway; triggers static serving
PORT	No	Express port — Railway sets this automatically
UPLOAD_DIR	No	Default ./uploads; set /app/uploads on Railway Volume
DB_PATH	No	Default ./data/vault.db; set /app/data/vault.db on Railway Volume

15. Acceptance Criteria

Local development

- npm run dev starts both servers; app loads at localhost:5173 with no errors
- All project data persists across page refreshes (SQLite)
- Fresh clone + npm install + npm run dev works with no manual database setup

Features

- Settings: changing font updates all UI text within 100ms
- Settings: changing colour theme updates all colours without page reload
- Chat: active project context fields appear in every Claude API system prompt
- Chat: @ mention dropdown lists all projects and injects reference block correctly
- Chat: mic button captures speech and fills input field
- Chat: speaker button reads assistant message aloud
- Search: Cmd+K opens palette; returns results across projects, files, messages
- Email: thread email arrives formatted with all messages intact
- Export chat PDF: all messages present, project name in header
- PDF upload: text extracted and shown in file card within 5 seconds
- PDF summary: Claude returns a concise summary of the document
- Gemini Nano: image generation produces a visible image in chat (Chrome only)
- Gemini Nano: image upload + describe returns a text description

Deployment

- git push origin main triggers Railway auto-deploy with no manual steps
- Railway health check GET /api/health returns 200 OK
- App loads on the Railway-provided URL after deploy
- Uploaded files and SQLite database persist across Railway redeploys (Volume attached)
- All environment variables set in Railway Variables — no .env file on server

— End of Document —